

FORGE

Framework for Orchestrated Requirements, Governance & Engineering

A structured software development system for OpenCode, Claude Code and Codex CLI

What is FORGE?

FORGE is a comprehensive methodology for AI-assisted software development that combines:

- **Structured progressive context** that prevents inconsistent decisions
- **Adaptive process** that calibrates ceremony to complexity
- **Persistent knowledge** that survives session boundaries

Multi-platform since v2.0 — OpenCode, Claude Code and Codex CLI — synthesising the best of BMAD Method and Speckit

The Problem

5 Critical Problems in AI-Assisted Development

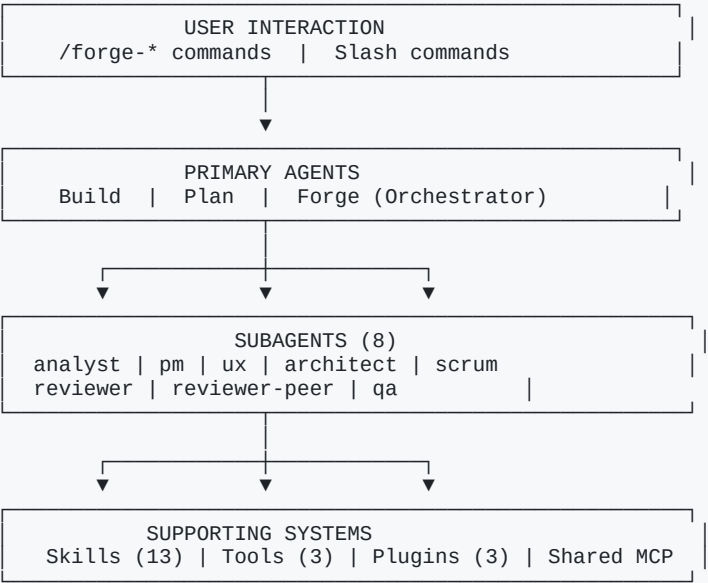
1. **Context Gap** - Agents in different sessions make conflicting decisions
2. **Ceremony Trap** - Too much process kills speed, too little kills quality
3. **Knowledge Evaporation** - Decisions disappear at the end of the session
4. **Consistency Entropy** - Teams of 15+ developers produce inconsistent code
5. **Illusion of Productivity** - Speed without direction is waste

The FORGE Solution

6 Core Principles

Principle	Description
Progressive Context Engineering	Each phase produces a document that becomes context for the next phase
Constitutional Governance	Immutable principles govern every decision
Adaptive Ceremony	Process depth scales with complexity
Adversarial Quality	Reviews MUST find problems
Persistent Knowledge	Decisions and lessons survive across sessions
Bidirectional Traceability	Requirements → code → tests (and back)

System Architecture



```
> v2.0: canonical artifacts in `.opencode/` are projected per platform –
> `.claude/` for Claude Code, `.codex/` for Codex CLI – with one shared MCP server

---

## v2.0: Multi-Platform by Design

| Platform | Vendor | Agents & Commands | Config | Instructions |
|-----|-----|-----|-----|-----|
| **OpenCode** | OpenCode | `.opencode/{agents,commands,skills}/` | `opencode.json` | `AGENTS.md` |
| **Claude Code** | Anthropic | `.claude/{agents,commands,skills}/` + hooks | `.claude/settings.json` | `CLAUDE.md` |
| **Codex CLI** | OpenAI | `.codex/{agents,commands}/` + `.agents/skills/` | `.codex/config.toml` | `AGENTS.md` |

**One installer auto-detects your runtime – same methodology everywhere**

```bash
npx tsx install-forge.ts /path/to/project # auto-detect
npx tsx install-forge.ts /path/to/project --platform=claude-code
```

## Track: Quick

**When:** Small feature, 1-5 tasks, < 1 day

```
/forge-quick "Add forgot password feature with 1-hour expiry token"
```

### Workflow:

1. ✓ Quick Spec - Conversation → `tech-spec.md`
2. ✓ Implement - Implement task by task
3. ✓ Test - Generate unit tests
4. ✓ Review - Adversarial self-review

**Output:** `.forge/specs/NNN-name/tech-spec.md` + code + tests

## Track: Feature

**When:** Medium feature, 5-20 tasks, 1-5 days

```
/forge-specify "Add OAuth2 authentication with Google and GitHub"
/forge-clarify # Resolve ambiguities
/forge-ux # UX design: personas, wireframes, accessibility
/forge-plan # Technical plan
/forge-analyze # Cross-validate spec vs plan
/forge-tasks # Task breakdown with dependencies
/forge-implement # Implement
/forge-review # Adversarial review (7 dimensions, dual-model)
```

**Output:** spec.md, design-spec.md, user-journey.md, plan.md, tasks.md, optional ADRs

# UX Phase: /forge-ux

**When:** Feature with user interface (web, mobile, design system)

/forge-ux "Login page with OAuth"

Artifact	Content	Focus
design-spec.md	ASCII wireframes, UI components, interaction specs	 wireframes ·  design system ·  responsive
user-journey.md	Personas, scenarios, user flows	 journeys ·  WCAG 2.1 AA

# Review: 7 Quality Dimensions

## The review covers all 7 dimensions:

Correctness	→ logic, edge cases
Security	→ vulnerabilities, injection
Performance	→ slow queries, memory leaks
Maintainability	→ readability, coupling
Constitution	→ adherence to principles
Test-Spec Coherence	→ tests cover spec requirements
UX Quality	→ accessibility, usability, design consistency

## UX issue examples:

- [HIGH] Button without `aria-label` — not accessible with screen reader
- [MEDIUM] Form without focus management — degraded UX after submit
- [LOW] Colour contrast 2.8:1 — below WCAG AA threshold (4.5:1)

# Track: Epic

**When:** Complex feature set, 20-50+ tasks, 1-4 weeks

/forge-brief	# Product brief
/forge-prd	# Full PRD
/forge-architecture	# Architecture + ADR
/forge-sprint	# Sprint planning
/forge-story	# Prepare story
/forge-implement	# Implement story
/forge-review	# Dual review (AI + Human)
/forge-retro	# Retrospective

**Output:** Brief, PRD, Architecture, Epic, Stories, Sprint Status, ADR

## Track: Product

**When:** New product, greenfield, 4+ weeks

```
/forge-init # Setup + Constitution
```

Then follows the Epic workflow with the addition of:

- **Constitution** - Immutable governance document
- **UX Phase** - `/forge-ux` for products with user interface

**Full output:** All Epic track documents + Constitution

# The 8 Specialized Subagents

Agent	Model	Role
forge-analyst	Sonnet 4.6	Exploration, research, scope detection
forge-pm	Opus 4.7	Requirements, spec, PRD, user stories
forge-ux	Sonnet 4.6	User journeys, wireframes, accessibility, design spec
forge-architect	Opus 4.7	Architecture, ADR, technical planning
forge-scrum	Sonnet 4.6	Sprint planning, story management
forge-reviewer	Opus 4.7	Adversarial review — Task A (7 dimensions)
forge-reviewer-peer	independent model	Adversarial review — Task B, independent
forge-qa	Sonnet 4.6	Test strategy, test generation

`/forge-review` launches **forge-reviewer + forge-reviewer-peer in parallel** and synthesises findings — *consensus findings* (both models) carry the highest priority

## Progressive Context Chain



### Each phase receives structured context from the previous phase

- Prevents inconsistent decisions
- Guarantees architectural alignment
- Accumulates knowledge over time

# Constitutional Governance

The **Constitution** is the highest-authority governance document

## 9 Standard Articles

1. **Core Principles** - Non-negotiable principles
2. **Technology Stack** - Approved tech stack
3. **Architecture Patterns** - Mandatory patterns
4. **Quality Standards** - Quality thresholds
5. **Security** - Security requirements
6. **Error Handling** - Error handling standards
7. **Naming & Conventions** - Naming conventions
8. **Testing Standards** - Required tests
9. **Operational Requirements** - Operational standards

# Persistent Knowledge Base

## 3 Types of Artefacts

```
.forge/knowledge/
├── adr/ # Architecture Decision Records
│ ├── 001-database-choice.md
│ └── 002-auth-strategy.md
├── decision-log.md # Session-extracted decisions
└── lessons-learned.md # Post-mortem insights
```

### Features:

- ✓ Created automatically by the `session-knowledge` plugin
- ✓ Persist across sessions
- ✓ Loaded as context every session
- ✓ Prevent repeated mistakes

# Adversarial Review

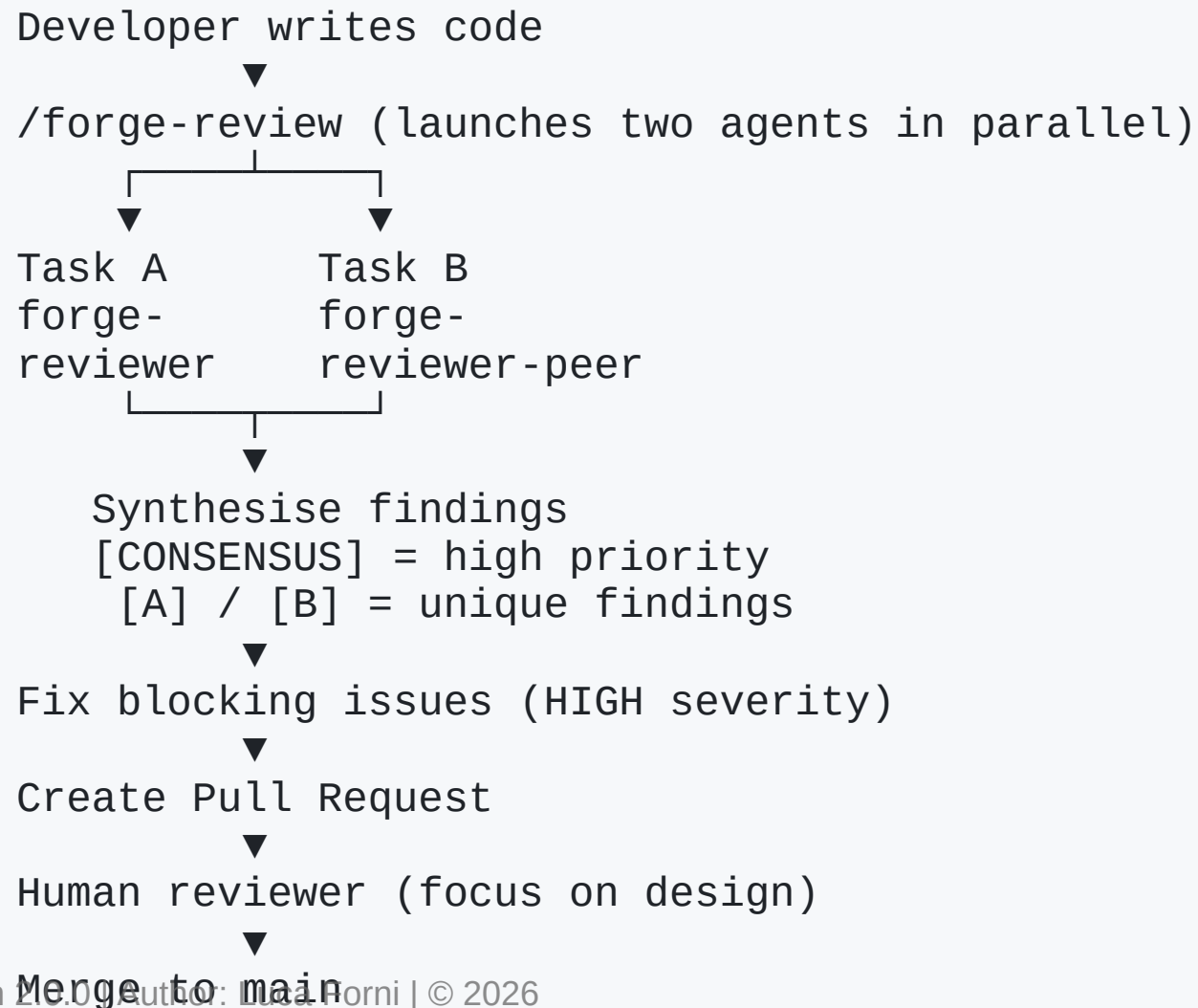
## Two Models in Parallel — Reviews that MUST Find Problems

forge-reviewer + forge-reviewer-peer examine across **7 dimensions**:

1. **Correctness** - Logic, edge cases, errors
2. **Security** - Vulnerabilities, input validation
3. **Performance** - Slow queries, memory leaks
4. **Maintainability** - Readability, coupling
5. **Constitution Compliance** - Adherence to principles
6. **Test-Spec Coherence** - Tests cover spec requirements
7. **UX Quality** - Accessibility (WCAG 2.1 AA), usability, design consistency

**Minimum 5 combined issues** across both models — *consensus findings* (flagged by both) carry the highest priority

## Dual Review Process



## Bidirectional Traceability

```
Requirement FR-001 (spec.md)
↓
Technical approach (plan.md, section 3.2)
↓
Task 2.1 (tasks.md)
↓
Source file (src/auth/login.ts)
↓
Test file (src/auth/__tests__/login.test.ts)
```

**Custom tool:** `/forge-analyze` generates traceability matrix

- Identifies unimplemented requirements
- Identifies code without requirements (orphan code)
- Verifies test coverage per requirement

# Brownfield Support

## Onboarding Existing Codebases

```
/forge-init # In existing project
```

### FORGE analyses:

- Project structure and languages
- Detected frameworks and architectural patterns
- Naming and organisation conventions
- Potential tech debt

### Output:

- Constitution auto-generated based on existing code
- **AGENTS.md** derived from `.eslintrc`, `tsconfig.json`, detected patterns

# 13 Dynamic Skills

Skill	Used by	Purpose
adversarial-review	forge-reviewer	Mandatory review protocol
advanced-elicitation	pm, architect	Deep analysis techniques
scope-detection	Forge orchestrator	Assess complexity, recommend track
test-strategy	forge-qa, Build	Adaptive test strategy per track
brownfield-analysis	forge-analyst	Existing codebase analysis
constitution-compliance	architect, reviewer	Verify constitution compliance
context-chain	All agents	Load correct upstream documents
ux-design	forge-ux	Generate user journeys, wireframes, a11y specs
ux-review	forge-reviewer	7th review dimension: UX quality & accessibility
frontend-pattern-library	forge-ux, Build	17 UI patterns (React + shadcn/ui)
data-presentation	forge-ux	Dashboards, visualisation, data storytelling

## Automatic Plugins (1/2)

### session-knowledge

- Automatically extracts decisions and lessons
- Appends to `decision-log.md` and `lessons-learned.md`
- Injects knowledge during context compaction

### pre-commit-gate

- Validates spec-code consistency before commit
- Verifies completed tasks and existing tests
- Advisory (does not block, but warns)

## Automatic Plugins (2/2)

### spec-watcher

- Monitors changes to `.forge/specs/`
- Detects inconsistencies with plan/tasks
- Suggests `/forge-analyze`

**Plugins automate governance and knowledge management without manual intervention**

# Benefits for Developers

Benefit	How FORGE Ensures It
Fast onboarding	Spec, plan, ADR provide full context
Quality on the first try	Structured context → better AI decisions
Less rework after review	AI review catches issues before human review
Clear requirements	/forge-clarify surfaces ambiguities before implementation
Confidence in decisions	Constitution and ADR validate choices

## Benefits for Teams

Benefit	How FORGE Ensures It
Consistent code	Constitution enforces patterns; all agents follow the same rules
Reduces architecture drift	ADRs prevent contradictory decisions
Efficient sprint management	<code>/forge-sprint</code> and <code>/forge-status</code> automate overhead
Effective retrospectives	<code>/forge-retro</code> produces actionable insights
Faster reviews	AI handles mechanical checks, humans focus on design
Smooth onboarding	New members read <code>.forge/</code> to understand the whole project
Safe parallel development	Architecture defines boundaries; ADRs prevent conflicts

## Benefits for Enterprise

Benefit	How FORGE Ensures It
Audit trail compliance	Constitution + ADR + decision log = complete rationale
Risk management	Spec includes risk sections; adversarial review finds issues early
Knowledge retention	When developers leave, decisions remain
Process standardisation	All teams use the same workflows, templates, protocols
Tech debt visibility	Brownfield analysis + traceability expose gaps
Quality metrics	Sprint velocity, issue count, spec completeness
Governance without bottleneck	Constitution + skills enforce standards automatically

# FORGE vs Alternatives

## vs No Methodology ("Vibe Coding")

	No Methodology	FORGE
Initial speed	Very fast	Moderate (planning overhead)
Speed over time	Slows down (debt accumulates)	Sustained (compounding knowledge)
Consistency	Random	Enforced
Rework	High (30-50%)	Low (< 15%)
Onboarding	Weeks	Days

# FORGE vs BMAD vs Speckit

	BMAD	Speckit	FORGE
Agent system	Personas (1 LLM)	None	Real subagents
Tracks	3	1	5
Governance	Weak	Constitution	Constitution + ADR + KB
Knowledge persistence	None	Per-branch	Cross-session KB
Review	Adversarial	None	Dual Adversarial
Brownfield	Limited	Limited	Structured
Platform	IDE-agnostic	Agent-agnostic	OpenCode-native

# When to Use FORGE

## Ideal Use Cases

Scenario	Track	Why
New SaaS from scratch	Product	Prevents tech debt
Enterprise feature	Epic	Prevents scope creep
Large codebase	Feature/Quick	Maintains consistency
Compliance-heavy	Epic/Product	Complete audit trail
Team with turnover	Any	Retains knowledge

# When NOT to Use FORGE

## Overkill Scenarios

Scenario	Better Alternative
One-off scripts or utilities	Write code directly
Tutorial / learning projects	Focus on learning, not process
Hackathon prototype (discard after)	Speed > structure
Projects with < 1 week total lifespan	The docs outlive the code
Safety-critical systems	FORGE + formal verification
Multi-team platform (50+ devs)	FORGE + enterprise program management

# The Cost of Structure

## Realistic Overhead

Activity	Overhead
Constitution	1-2 hours (once)
Spec (Feature)	20-30 min
Plan	15-20 min
Analyze	5 min
Review	10-15 min
Retro	15-20 min/sprint

**Total per feature: ~1-1.5 hours**

## Break-Even Analysis

### Without FORGE:

Implementation: 3 days

Rework after review: 0.5 days (frequent)

Rework after production: 1 day (occasional)

Total: 3.8 days

### With FORGE:

Planning + review: 0.2 days

Implementation: 2.5 days (better context = faster)

Rework after review: 0.1 days (AI review catches issues)

Rework after production: 0.1 days (rare with dual review)

Total: 2.9 days

**Break-even typically at the 2nd–3rd feature of the project**

## Compound Effect

Session 1:      You pay the constitution cost → high overhead, zero benefit  
Session 10:     Constitution prevents a bad decision → benefit > costs  
Session 50:     New dev productive in hours thanks to .forge/  
Session 200:    Auditor asks encryption rationale → ADR-012 → 1 day vs 1 week

### Structure has decreasing costs and compounding returns

The question is not "can we afford the overhead?" but  
**"can we afford NOT to have it?"**

# Technical Components

## 9 Agents | 24 Commands | 13 Skills | 20 Templates

```
.opencode/ # Canonical source (projected per platform)
├── agents/ # 8 specialized subagents + forge orchestrator
├── commands/ # 24 slash commands
├── skills/ # 13 dynamic skills
├── tools/ # 3 custom tools (via shared MCP server)
├── plugins/ # 3 automation plugins
└── templates/ # 20 document templates
.claude/ # Claude Code projection (agents, commands, hooks)
.codex/ # Codex CLI projection (agents, commands)
```

**One shared MCP server ( `validate-spec`, `trace-requirements`, `sprint-status` )  
serves all three runtimes**

# Quick Start

## 1. Installation (auto-detects your runtime)

```
From the FORGE repo – OpenCode, Claude Code and Codex CLI detected automatically
npx tsx install-forge.ts /path/to/your/project

Or target a platform explicitly
npx tsx install-forge.ts /path/to/your/project --platform=claude-code
```

## 2. Verify

```
cd your-project
opencode
> /forge-help
```

## 3. First Workflow (Quick)

```
> /forge-quick "Add health check endpoint returning app version"
```

# Directory Structure



# Model Strategy

## Differentiation by Cognitive Demand

Model	When	Agents
Opus 4.7	Deep reasoning, architectural decisions, adversarial review	forge-pm, forge-architect, forge-reviewer
Sonnet 4.6	Speed, good-enough reasoning, analysis, sprint mgmt	Forge, forge-analyst, forge-scrum, forge-qa
Independent model	Independent adversarial review (second opinion)	forge-reviewer-peer

Models assigned centrally in `opencode.json` — no model directives in agent frontmatter

# Incremental Adoption Path

## No Need to Adopt Everything at Once

**Week 1-2:** Hotfix + Quick only

- Get familiar with the workflow without changing existing process

**Week 3-4:** Feature track for a medium feature

- Experience the full spec-plan-implement-review cycle

**Month 2:** Epic track for a larger initiative

- Add sprint management

**Month 3+:** Evaluate Product track and constitutional governance

# Essential Commands

## Quick Reference Card

Tracks:	/forge-hotfix	Bug fix, 1 file, < 30 min
	/forge-quick	Small feature, 1-5 tasks
	/forge-specify	Medium feature (start here)
	/forge-brief	Large epic
	/forge-init	New product
Feature Flow:	specify → clarify → ux → plan → analyze → tasks → implement → review	
UX Commands:	/forge-ux	User journeys, wireframes, a11y
	/forge-wireframe	ASCII wireframe generation
Status:	/forge-status	Sprint dashboard
	/forge-help	Context-aware help
Knowledge:	/forge-adr	Create ADR
	/forge-retro	Sprint retrospective

## Use Case: OAuth Feature

```
1. Specify
/forge-specify "Add OAuth2 auth with Google and GitHub"

2. Clarify ambiguities
/forge-clarify

3. UX Design
/forge-ux # login wireframe, user journey, a11y

4. Technical plan
/forge-plan

5. Validate consistency
/forge-analyze

6. Task breakdown
/forge-tasks

7. Implement
/forge-implement

8. Adversarial review (7 dimensions, dual-model)
/forge-review
```

# Team Workflow

## Multi-Developer Feature Development

Developer A: Epic 1, Stories S001-S004 (Core Payments)  
Developer B: Epic 2, Stories S001-S003 (Subscriptions)  
Developer C: Epic 3, Stories S001-S003 (Webhooks)

### Shared artefacts (via git):

- `.forge/constitution.md` - Everyone follows the same principles
- `.forge/architecture/` - Consistent technical decisions
- `.forge/knowledge/adr/` - Everyone sees all decisions
- `.forge/sprints/` - Scrum master updates centrally

### Conflict prevention:

- Architecture defines module boundaries

# Sprint Ceremonies with FORGE (1/2)

## Sprint Planning

```
/forge-sprint
→ Review velocity
→ Select stories from backlog
→ Assign to developer
→ Update sprint-status.yaml
```

## Daily Standup

```
/forge-status
→ See assigned stories and status
→ Flag blockers
```

## Sprint Ceremonies with FORGE (2/2)

### Sprint Retrospective

```
/forge-retro
→ Automatic velocity calculation
→ Lessons learned → knowledge base
```

# Knowledge Base Maintenance

Task	Frequency	Owner
Review decision-log, promote to ADR	Weekly	Tech Lead
Archive stale lessons-learned	Per sprint	Scrum Master
Review ADR staleness	Monthly	Architect
Verify constitution accuracy	Quarterly	Tech Lead + PM

**15 minutes/week for KB maintenance**

# Customisation

## FORGE is Fully Customisable

- **Constitution Template** - Adapt articles to your domain
- **Agent System Prompts** - Tune for team style
- **Skills** - Add domain-specific reasoning
- **Templates** - Modify document structure
- **Commands** - Create custom workflows
- **Plugins** - Project-specific automation

**See:** `.opencode/docs/FORGE-CUSTOMIZATION.md`

# CI/CD Integration

## Quality Gates on Every Change

Gate	What it does
CI matrix	Tests on Node 20 / 22 / 24, blocks merge on red
CodeQL	Static security analysis (JS/TS, security-extended)
Scorecard	Supply-chain posture (incl. SHA-pinned actions)
Dependabot	Weekly updates (npm + Actions, grouped, auto-PR)
Branch protection	PR + code-owner review, linear history, no force-push
Pre-commit gate	Advisory: tasks done, tests exist, constitution verified

Every PR gets dual-model adversarial review before human review

# Advanced Elicitation Techniques

## 6 Deep Analysis Techniques

1. **Pre-mortem Analysis** - Imagine the feature failed — what went wrong?
2. **First Principles Thinking** - Decompose into fundamental components
3. **Red Team / Blue Team** - Attacker vs defender
4. **Socratic Questioning** - Deep questions about assumptions
5. **Constraint Removal** - What if we had no constraints?
6. **Inversion Analysis** - How would we guarantee it FAILS?

Used by `forge-pm` and `forge-architect` for spec and architecture analysis

# Scope Detection

## Automatic Complexity Assessment

### 7 Factors:

- Files affected (1-2 → 50+)
- Estimated tasks (1 → 50+)
- New dependencies (0 → Stack decision)
- Schema changes (None → Full design)
- API surface changes
- Cross-module impact
- Need for new patterns

**Output:** Structured JSON with recommended track + reasoning

# Adaptive Test Strategy

## Coverage Requirements per Track

Track	Required Tests
Hotfix	Regression test for the bug only
Quick	Unit tests for new/modified code
Feature	Unit + Integration tests
Epic	Unit + Integration + E2E
Product	Unit + Integration + E2E + Performance benchmarks

Defined by the test-strategy skill, executed by forge-qa

## 3-Level Harness Tests

How FORGE tests itself — same pyramid for any project:

Level	What	Where	Secrets?
1. Unit	74 Vitest tests (installer, MCP, validators)	Every PR	None
2. Projection	Install layout per platform (opencode/claude/codex)	Every PR	None
3. Smoke	Real CLI runs ( <code>opencode run</code> , <code>codex exec</code> ) vs LLM gateway	Weekly + manual	Yes (env-gated)

Live smokes assert real tool-calls; missing keys/CLI → skip, never fail

# ADR (Architecture Decision Records)

## When to Create an ADR

- Database, framework, major library choice
- API style decision (REST vs GraphQL vs gRPC)
- Defining pattern (event-driven vs synchronous)
- Trade-off (consistency vs availability)
- Any decision someone will question later

## Format

```
Context: Why is this decision needed?
Options: What alternatives were considered?
Decision: What was chosen and why?
Consequences: Positive, negative, neutral effects
Constitution Alignment: Which articles does it support?
```

# Retrospectives

## /forge-retro Output

Sprint 2 Retrospective

=====

Velocity: 32 pts (planned: 34, previous: 28)

What went well:

- Stripe integration straightforward thanks to clear ADRs
- Task parallelism markers saved time

What could improve:

- Webhook testing required manual Stripe CLI setup (not in spec)
- OAuth PR took 2 days human review (bottleneck)

Action items:

- Add webhook test setup instructions to spec template
- Rotate PR reviewer to avoid single-person bottleneck

## Traceability Matrix Example

```
Requirement FR-001 (Login with OAuth)
 → Plan Section 3.2 (OAuth flow implementation)
 → Task 2.1 (Implement OAuth strategy)
 → src/auth/oauth-strategy.ts ✓
 → src/auth/__tests__/oauth-strategy.test.ts ✓

Requirement FR-002 (Account linking)
 → Plan Section 3.3 (Link accounts)
 → Task 3.1 (Link endpoint) → [NOT IMPLEMENTED] ⚠

Requirement NFR-001 (P95 < 200ms)
 → [NO TASK] → [NOT IMPLEMENTED] ⚠
```

Generated by the **trace-requirements** tool

# Success Metrics

## How to Measure FORGE Success

Metric	Target	How Measured
Rework rate	< 15%	Issues after PR merge / Total PRs
Onboarding time	< 3 days	New dev to first meaningful commit
Architecture drift incidents	0	ADR violations detected
Knowledge retention	100%	Documented decisions / Total decisions
Sprint velocity	+20% after 3 sprints	Story points delivered
Code consistency	> 90%	Constitution compliance score

## Resources & Documentation

### Full Documentation in `.opencode/docs/`

- **FORGE-GUIDE.md** - Complete usage guide
- **FORGE-PHILOSOPHY.md** - Principles and benefits
- **FORGE-PROJECT-PLAN.md** - System architecture
- **FORGE-CUSTOMIZATION.md** - How to customise
- **FORGE-DECISIONS.md** - Methodology decision records

### Community & Support

- GitHub: `lucaforni/forge`
- OpenCode Docs: `https://opencode.ai/docs`

# Future Roadmap

## Shipped in v2.0

- **Multi-platform** - OpenCode + Claude Code + Codex CLI via projection installer
- **CI/CD pipeline** - Matrix CI, CodeQL, Scorecard, Dependabot, branch protection
- **Shared MCP server** - Custom tools on all three runtimes
- **Harness test strategy** - Unit + projection + live CLI smokes

## Possible Evolutions

- **AI Pair Programming Mode** - Forge assists in real-time during coding
- **Multi-repo Support** - FORGE coordination across microservices
- **Custom Domain Templates** - Pre-built constitutions for fintech, healthcare, gaming
- **Integration with Project Management Tools** - Jira, Linear, Azure DevOps
- **VS Code Extension** - Quick command access

# Key Takeaways

## Why FORGE Changes the Game

1. **Structured context** prevents inconsistencies across AI sessions
2. **5 workflow tracks** adapt the process to complexity
3. **UX-first design** includes user journeys, wireframes and accessibility in the workflow
4. **Constitutional governance** ensures a uniform quality bar
5. **Persistent knowledge base** eliminates knowledge loss
6. **Adversarial review** catches issues before production (7 dimensions, dual-model)
7. **Multi-platform** runs the same methodology on OpenCode, Claude Code and Codex CLI
8. **Brownfield support** onboards existing codebases
9. **Team-ready** supports 15+ developers with parallel development
10. **CI-enforced quality** (matrix tests, CodeQL, Dependabot) + 3-level harness tests

# Get Started Today (1/2)

## 3 Steps to Try FORGE

### 1. Setup (5 minutes)

```
npx tsx forge/install-forge.ts /path/to/your/project
cd /path/to/your/project && opencode
```

### 2. First Task (5 minutes)

```
> /forge-quick "Your first small feature"
```

## Get Started Today (2/2)

### 3 Steps to Try FORGE

#### 3. Evaluate Results

- Is the produced spec clear?
- Is the implementation clean?
- Were tests generated correctly?

**If yes → Continue with Feature track**

**If no → Contact support for tuning**

# Questions?

---

## Contacts:

- Documentation: `.opencode/docs/`
- GitHub Issues: `lucaforni/forge`
- OpenCode Docs: `https://opencode.ai/docs`

# Thank You!

---

## FORGE

Framework for Orchestrated Requirements, Governance & Engineering

Enterprise software development with AI — structured and sustainable

**Author:** Luca Forni

 [linkedin.com/in/lucaforni](https://www.linkedin.com/in/lucaforni)

 [github.com/lucaforni](https://github.com/lucaforni)

*Version 2.0.0 | MIT License | 2026*